

Expt 3: Write a program to implement data link layer framing method checksum

Aim:

To write a Python program to implement the Checksum error detection technique used in the Data Link Layer for detecting errors during data transmission.

Theory:

Checksum is an error-detection technique used in data communication. It helps detect errors in the transmitted data by appending a calculated value (checksum) to the data before transmission.

Basic Concept:

Divide the binary data into equal-sized blocks (e.g., 8 bits).

Add all the blocks using binary addition with carry.

Take the 1's complement of the result.

Transmit all blocks + the checksum.

At the receiver, add all the blocks including checksum.

- If the result is all 1s, then no error.
- If any 0 is present, then error occurred.

Algorithm:

1. At the Tx:

Input: N binary data blocks of equal length

Output: Checksum

1. Initialize checksum = first data block
2. For each remaining block:
 - a. Add the block to checksum using binary addition with carry
3. Take 1's complement of the final sum
4. Append this checksum to the data blocks for transmission

2. At the Rx:

Input: Received data blocks + checksum

Output: Valid or Invalid transmission

1. Initialize sum = first data block
2. For each remaining block including checksum:

a. Add the block to sum using binary addition with carry

3. If the final result contains only 1s:

Transmission is Valid

Else:

Error in Transmission

Program:

```
# Function to add two binary strings with carry
def binary_addition(a, b):
    max_len = max(len(a), len(b))
    a = a.zfill(max_len)
    b = b.zfill(max_len)

    result = ''
    carry = 0

    for i in range(max_len - 1, -1, -1):
        total = carry + int(a[i]) + int(b[i])
        result = str(total % 2) + result
        carry = total // 2

    if carry:
        result = '1' + result

    # If overflow occurs, add the overflow back (1's complement addition)
    if len(result) > max_len:
        result = binary_addition(result[1:], result[0])

    return result[-max_len:] # Trim overflow

# Function to find 1's complement
def ones_complement(binary_str):
    return ''.join('1' if b == '0' else '0' for b in binary_str)

# Function to calculate checksum
def calculate_checksum(data_blocks):
    checksum = data_blocks[0]
    for block in data_blocks[1:]:
        checksum = binary_addition(checksum, block)
    checksum = ones_complement(checksum)
    return checksum

# Function to verify checksum
def verify_checksum(data_blocks, received_checksum):
```

```

total = data_blocks[0]
for block in data_blocks[1:]:
    total = binary_addition(total, block)
total = binary_addition(total, received_checksum)
return all(bit == '1' for bit in total)

# Main
if __name__ == "__main__":

    # Sender Side
    sender_data = ["11010101", "10101010", "11110000"]

    print("----- SENDER SIDE -----")
    print("Sender Data Blocks:")
    for block in sender_data:
        print(block)

    checksum = calculate_checksum(sender_data)
    print("Calculated Checksum:", checksum)

    # Receiver Side
    print("\n----- RECEIVER SIDE -----")
    print("Enter the received data blocks:")

    received_data = []
    for i in range(len(sender_data)):
        block = input(f"Enter Block {i+1}: ")
        received_data.append(block)

    received_checksum = input("Enter the received checksum: ")

    print("\nChecking...")

    if verify_checksum(received_data, received_checksum):
        print("Data received correctly. No Error Detected.")
    else:
        print("Error in Data Transmission Detected.")

```

Output

Applications:

1. Detects errors during data transmission in computer networks.
2. Used in IP, TCP, and UDP protocols for error detection.
3. Used in wireless, IoT, and embedded system communications.
4. Improves the reliability of digital communication systems.